

28. preflight 判定ロジック

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	4
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	5
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 class、独自クラス、インスタンス、self、継承	6
6.7 先頭アンダースコア、__init__、名前付けの作法	7
6.8 デコレータと @dataclass	7
6.9 制御、状態、入力、モデル予測制御 MPC	7
7 関数・クラスを上から順に解説	8
7.1 L7 クラス FreshnessResult	8
7.2 L14 クラス CommandGateResult	9
7.3 L19 関数 evaluate_freshness	9
7.4 L43 関数 evaluate_command_gate	11

8 処理の流れ	12
9 このファイルの位置づけ	12

1 対象

項目	内容
ファイル	mpc_solarcar/solar_preflight_logic.py
ソース SHA-256	2af5eb32f0e80a828a2fde431fe670d653b1aca06ee4c000690c8e503aa684d9
種別	Python
区分	runtime helper
役割	計測鮮度や command gate の純判定を Node 本体から切り出したロジック関数群。
起動文脈	preflight と speed bridge の共通判定層。

2 このファイルを読む理由

計測鮮度や command gate の純判定を Node 本体から切り出したロジック関数群。

- evaluate_freshness と evaluate_command_gate が中心。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/solar_preflight_node.py
- mpc_solarcar/speed_command_bridge_node.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
@dataclass(frozen=True)
class FreshnessResult:
    state: str
    health: float
    diagnostic: str

@dataclass(frozen=True)
class CommandGateResult:
    allowed: bool
    reason: str
```

```
def evaluate_freshness(
    *,
    elapsed_sec: float,
    ages_sec: dict[str, float | None],
    required: tuple[str, ...],
    timeout_sec: float,
    startup_grace_sec: float,
```

5 主要構造の読み取り

主要クラスは `FreshnessResult`, `CommandGateResult`。主要関数は `evaluate_freshness`, `evaluate_command_gate`。

5.1 主要クラス・関数

行	種別	名前	補足
7	class	<code>FreshnessResult</code>	
14	class	<code>CommandGateResult</code>	
19	function	<code>evaluate_freshness</code>	
43	function	<code>evaluate_command_gate</code>	

5.2 ファイルを上から読んだときの定義順

行	内容
7	クラス <code>FreshnessResult</code> を定義する。
14	クラス <code>CommandGateResult</code> を定義する。
19	関数 <code>evaluate_freshness</code> を定義する。
43	関数 <code>evaluate_command_gate</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from __future_</code> <code>_import annotations</code>	型ヒントの遅延評価を有効にし、自己参照型や <code>forward reference</code> を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>from dataclasses import dataclass</code>	<code>dataclasses</code> から <code>dataclass</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L6, L13。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼び
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 class、独自クラス、インスタンス、self、継承

クラスは、保持するデータと、そのデータを扱う関数を一つの型としてまとめる設計単位である。「独自クラス」とは Python や ROS 2 が最初から用意した型ではなく、このプロジェクトが目的に合わせて定義した新しい型を指す。

‘class MPCNode(Node)’は、rclpy の Node を基底クラスとして継承し、Node が持つ publisher、subscription、timer、parameter などの機能に MPC 固有機能を追加する。丸括弧内は関数引数ではなく基底クラス指定である。

‘MPCNode()’はクラスオブジェクトを呼び出して新しいインスタンスを作る。Python は新しい実体を用意し、その実体を第 1 引数 self として ‘__init__’へ渡す。‘self’は予約語ではなく慣習的な名前だが、この慣習を守ることでコードの意味が共有される。

```
class Counter:
    def __init__(self):
        self.value = 0

    def increment(self):
        self.value += 1

counter = Counter()
counter.increment()
```

‘super().__init__(...)’は継承元の初期化処理を呼ぶ。MPCNode ではこれを省くと ROS ノードとして必要な内部実体を作られず、‘create_publisher’などを正しく使えない。

根拠資料

- [Python 公式チュートリアル: Classes](#)
- [ROS 2 Humble 公式: Understanding nodes](#)

6.7 先頭アンダースコア、__init__、名前付けの作法

‘self_step_solar’の先頭 1 個のアンダースコアは「クラス内部で使う実装詳細」という慣習であり、アクセスを強制禁止する機構ではない。外部コードから呼べるが、公開 API として依存しない意思表示である。

‘__init__’の前後 2 個のアンダースコアは Python が定めた特殊メソッド名である。クラス生成時に自動で呼ばれる。任意の名前に前後 2 個を付けて独自仕様を作ることは避ける。

‘_’で始まるローカル変数は、未使用または外部へ見せない意図を示す場合がある。例えば ‘_base_csv’は戻り値として受け取る必要はあるが、その後の処理では使わないことを読者へ示す。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.8 デコレータと @dataclass

‘@ 名前’は、直後に定義した関数またはクラスを別の関数へ渡し、その結果で定義名を置き換えるデコレータ構文である。‘@Class’という一般構文があるのではなく、‘@dataclass’など実際のデコレータ名を書く。

‘@dataclass’は型注釈付きフィールドから ‘__init__’、‘__repr__’、比較処理などを自動生成する。物理パラメータや解析結果のように、名前付きデータを一まとまりで運ぶ用途に向く。

```
from dataclasses import dataclass

@dataclass
class State:
    soc: float
    temperature_c: float

state = State(soc=0.8, temperature_c=30.0)
```

自動生成は検証を自動で保証しない。単位、許容範囲、相互依存制約は ‘__post_init__’ や別の検証関数で確認する必要がある。

根拠資料

- [Python 公式ライブラリ: dataclasses](#)
- [Python 公式チュートリアル: Classes](#)

6.9 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’ と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L7 クラス FreshnessResult

- 行範囲: L7–L10
- 所属: module 直下
- 定義: `FreshnessResult(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `@` で始まる行は、定義したクラスを加工する `decorator` である。

上から順の処理

1. `state` にを代入する。
2. `health` にを代入する。
3. `diagnostic` にを代入する。

代表コード断片

```
class FreshnessResult:
    state: str
    health: float
    diagnostic: str
```


7.2 L14 クラス CommandGateResult

- 行範囲: L14–L16
- 所属: module 直下
- 定義: `CommandGateResult(bases=None)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: 特に明示されていない。
- 戻り値の要点: 戻り値が無いが、`return` 文が明示されていない。
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- `class` 文は新しい型を定義する。丸括弧内は継承する基底クラスである。
- `@` で始まる行は、定義したクラスを加工する decorator である。

上から順の処理

1. `allowed` にを代入する。
2. `reason` にを代入する。

代表コード断片

```
class CommandGateResult:
    allowed: bool
    reason: str
```

7.3 L19 関数 evaluate_freshness

- 行範囲: L19–L40
- 所属: module 直下
- 定義: `evaluate_freshness(*, elapsed_sec: float, ages_sec: dict[str, float | None], required: tuple[str, ...], timeout_sec: float, startup_grace_sec: float) -> FreshnessResult`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `FreshnessResult`, `float`, `get`, `join`
- 戻り値の要点: `FreshnessResult('RUNNING', 1.0, 'solar telemetry and planner inputs are fresh')` / `FreshnessResult('STARTING', 0.25, 'waiting for required solar telemetry')` / `FreshnessResult('DEGRADED', 0.2, 'missing: ' + ', '.join(missing))` / `FreshnessResult('DEGRADED', 0.4, 'stale: ' + ', '.join(stale))`
- 主なローカル代入名: `missing`, `name`, `stale`
- 読み取る主なインスタンス属性: 特になし。

- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 3、ループ 0、try 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。
- 内包表記は、反復・条件・値生成を一つの式で記述して `collection` を作る。

上から順の処理

1. 条件 `elapsed_sec < startup_grace_sec` を判定し、真なら内部処理を行う。
2. `FreshnessResult('STARTING', 0.25, 'waiting for required solar telemetry')` を返す。
3. `missing` に `[name for name in required if ages_sec.get(name) is None]` の結果を代入する。
4. `stale` に `[name for name in required if ages_sec.get(name) is not None and float(ages_sec[name]) > timeout_sec]` の結果を代入する。
5. 条件 `missing` を判定し、真なら内部処理を行う。
6. `FreshnessResult('DEGRADED', 0.2, 'missing: ' + ', '.join(missing))` を返す。
7. 条件 `stale` を判定し、真なら内部処理を行う。
8. `FreshnessResult('DEGRADED', 0.4, 'stale: ' + ', '.join(stale))` を返す。
9. `FreshnessResult('RUNNING', 1.0, 'solar telemetry and planner inputs are fresh')` を返す。

代表コード断片

```
def evaluate_freshness(
    *,
    elapsed_sec: float,
    ages_sec: dict[str, float | None],
    required: tuple[str, ...],
    timeout_sec: float,
    startup_grace_sec: float,
) -> FreshnessResult:
    if elapsed_sec < startup_grace_sec:
        return FreshnessResult("STARTING", 0.25, "waiting for required solar telemetry")

    missing = [name for name in required if ages_sec.get(name) is None]
    stale = [
        name
        for name in required
        if ages_sec.get(name) is not None and float(ages_sec[name]) > timeout_sec
    ]
    if missing:
        return FreshnessResult("DEGRADED", 0.2, "missing: " + ", ".join(missing))
    if stale:
        return FreshnessResult("DEGRADED", 0.4, "stale: " + ", ".join(stale))
    return FreshnessResult("RUNNING", 1.0, "solar telemetry and planner inputs are fresh")
```

7.4 L43 関数 `evaluate_command_gate`

- 行範囲: L43–L68
- 所属: module 直下
- 定義: `evaluate_command_gate(*, elapsed_sec:float, speed_input_age_sec:float|None, system_state:str, system_state_age_sec:float|None, startup_hold_sec:float, input_timeout_sec:float, system_state_timeout_sec:float, require_system_running:bool)-> CommandGateResult`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `CommandGateResult`, `max`, `str`, `strip`, `upper`
- 戻り値の要点: `CommandGateResult(True, 'ok')` / `CommandGateResult(False, 'startup_hold')` / `CommandGateResult(False, 'missing_speed_command')` / `CommandGateResult(False, 'stale_speed_command')`
- 主なローカル代入名: 特になし。
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 7、ループ 0、`try` 0

この定義を読むための Python 構文

- ‘->’以後は戻り値の型注釈であり、実行時の値を自動変換する命令ではない。

上から順の処理

1. 条件 `elapsed_sec < max(0.0, startup_hold_sec)` を判定し、真なら内部処理を行う。
2. `CommandGateResult(False, 'startup_hold')` を返す。
3. 条件 `speed_input_age_sec is None` を判定し、真なら内部処理を行う。
4. `CommandGateResult(False, 'missing_speed_command')` を返す。
5. 条件 `speed_input_age_sec > max(0.0, input_timeout_sec)` を判定し、真なら内部処理を行う。
6. `CommandGateResult(False, 'stale_speed_command')` を返す。
7. 条件 `not require_system_running` を判定し、真なら内部処理を行う。
8. `CommandGateResult(True, 'ok_without_system_gate')` を返す。
9. 条件 `system_state_age_sec is None` を判定し、真なら内部処理を行う。
10. `CommandGateResult(False, 'missing_system_state')` を返す。
11. 条件 `system_state_age_sec > max(0.0, system_state_timeout_sec)` を判定し、真なら内部処理を行う。
12. `CommandGateResult(False, 'stale_system_state')` を返す。
13. 条件 `str(system_state).strip().upper() != 'RUNNING'` を判定し、真なら内部処理を行う。
14. `CommandGateResult(False, 'system_not_running')` を返す。
15. `CommandGateResult(True, 'ok')` を返す。

代表コード断片

```
def evaluate_command_gate(*,
    elapsed_sec: float,
    speed_input_age_sec: float | None,
    system_state: str,
    system_state_age_sec: float | None,
    startup_hold_sec: float,
    input_timeout_sec: float,
    system_state_timeout_sec: float,
    require_system_running: bool,
) -> CommandGateResult:
    if elapsed_sec < max(0.0, startup_hold_sec):
        return CommandGateResult(False, "startup_hold")
    if speed_input_age_sec is None:
        return CommandGateResult(False, "missing_speed_command")
    if speed_input_age_sec > max(0.0, input_timeout_sec):
        return CommandGateResult(False, "stale_speed_command")
    if not require_system_running:
        return CommandGateResult(True, "ok_without_system_gate")
    if system_state_age_sec is None:
        return CommandGateResult(False, "missing_system_state")
    if system_state_age_sec > max(0.0, system_state_timeout_sec):
        return CommandGateResult(False, "stale_system_state")
    if str(system_state).strip().upper() != "RUNNING":
        return CommandGateResult(False, "system_not_running")
    return CommandGateResult(True, "ok")
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/solar_preflight_logic.py` を **runtime helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。